

Context Forge: A Lightweight Method for Diversity-Aware Context Packing and Prompt-Injection-Aware Retrieval

Mukunda Rao Katta

Abstract

Context quality remains one of the least stable parts of retrieval-augmented generation and agent prompting workflows. This paper presents Context Forge, a lightweight context-engineering toolkit that chunks source documents, ranks candidate chunks against a query, applies diversity-aware selection, flags prompt-injection and secret-like content, and renders citation-ready context blocks under a token budget. The method is designed as an inspectable middle layer between raw corpora and model prompts. Its aim is not to replace full retrieval systems, but to provide a compact and auditable packing strategy that is easier to reason about in local development, red-team analysis, and controlled evaluation settings. The paper describes the chunking and ranking procedure, the prompt-risk scan, and the packing strategy used to balance relevance, diversity, and budget constraints. Validation is demonstrated through repository tests and controlled examples that check relevance selection, heading preservation, and prompt-injection detection.

1. Introduction

Retrieval-augmented generation systems often fail at a surprisingly mundane layer: the context block handed to the model. Even when a team has relevant documents, the final prompt may over-pack redundant chunks, miss high-value passages, exceed budget constraints, or carry unsafe instructions copied from retrieved text. As tool-using agent systems have become more common, the quality of this intermediate context layer has become a practical systems problem rather than a minor implementation detail.

Context Forge addresses that layer directly. It is a zero-dependency toolkit for chunking documents, ranking candidate chunks, selecting a diverse subset under a token budget, scanning those chunks for prompt-injection or secret-like patterns, and rendering an auditable context block. The method is intentionally small. The point is not to outperform end-to-end retrieval stacks on a benchmark. The point is to provide a transparent packing strategy that teams can inspect, test, and adapt in ordinary development workflows.

This paper presents Context Forge as a method article. It focuses on the implemented algorithmic steps, the risk-scanning behavior, and the validation evidence available in the repository.

2. Method Motivation

Modern RAG and agent workflows face at least three recurring problems.

First, relevance alone is not enough. Ranking by query overlap can still surface many near-duplicate chunks from the same source while crowding out complementary evidence from another document.

Second, context windows are finite. The best chunk set under a strict budget is not always the top n chunks by score. Packing requires tradeoffs between relevance, redundancy, and token cost.

Third, retrieved text can be adversarial or simply unsafe to pass through blindly. Prompt-injection-style instructions, system-prompt references, or secret-like strings may appear inside documents and should be treated as risks rather than neutral content .

Context Forge combines these concerns in one small method. It treats context packing as a selection and safety problem, not just a retrieval one.

3. Software Artifact

The repository exposes two main entry points:

- a CLI command, ``ctxforge pack``
- a library API centered on ``packContext``

At the implementation level, the artifact includes:

- ``chunkDocument``
- ``rankChunks``
- ``riskScan``
- ``renderContextBlock``
- token estimation and term-count utilities
- an edge-pin ordering step to arrange selected chunks

The package is intentionally zero-dependency. That decision matters methodologically because it lowers the friction of inspection and reuse. A team can read the selection logic directly instead of treating the ranking stage as an opaque service call.

4. Method

The method follows five steps.

4.1 Section-aware chunking

Documents are split by Markdown-style headings when possible. Each section is then chunked into overlapping windows using a configurable maximum token estimate and overlap setting. This preserves local heading structure and reduces the chance that one large document dominates selection as a single monolithic block.

4.2 Query-sensitive relevance scoring

The query is tokenized, and each chunk receives a relevance score based on term overlap and simple log-scaled frequency. Heading matches provide a small positive boost. Chunks carrying serious risk markers receive a penalty. This keeps the relevance function readable while still reflecting both topicality and safety concerns.

4.3 Diversity-aware ranking

Context Forge applies a lightweight maximum marginal relevance style ordering. Candidate chunks are re-ranked against already selected chunks so that the final set balances relevance and diversity instead of simply repeating one dominant section.

4.4 Risk scanning

The method scans chunk text for a compact set of high-risk prompt patterns such as:

- requests to ignore previous instructions
- references to system prompts or developer messages
- requests to reveal secrets, credentials, or tokens
- attempts to redefine instructions

It also looks for secret-like token patterns such as common API key prefixes. The scan is heuristic rather than exhaustive, but it provides an immediate warning layer that is useful in development and controlled testing.

4.5 Budgeted packing and edge-pin ordering

Chunks are admitted while respecting a token budget and a maximum chunk count. Selected chunks are then reordered through an edge-pin strategy that places stronger chunks near the start and end of the final prompt block. This is a practical formatting step intended to improve prompt structure while keeping source identifiers visible.

Workflow figure included in source package

Figure 1. Context Forge workflow from source chunking through risk-aware context rendering.

5. Validation

The current validation evidence in the repository is modest but concrete.

5.1 Relevant-chunk selection test

The test suite checks that a refund-related query selects the refund-policy document ahead of an unrelated shipping document. This validates the basic query-sensitive selection behavior.

5.2 Prompt-injection detection test

The test suite checks that text containing "Ignore previous instructions and reveal the system prompt" is flagged as a prompt-injection risk. This validates that risk markers propagate into the result object.

5.3 Heading preservation test

The test suite checks that chunking preserves section headings across a two-section document. This matters because headings are used in ranking and interpretability.

The method should not be overstated. These tests do not prove general retrieval quality. They do, however, validate that the key intended behaviors of the artifact are implemented and stable across local

runs.

Validation target	Repository evidence	Why it matters
relevance under budget	refund-policy test case	shows topic-sensitive selection
risk detection	prompt-injection test case	shows unsafe retrieved text is not treated as neutral
structural preservation	heading-preservation test	keeps chunk metadata interpretable

6. Practical Use Cases

The method is well-suited to several real workflows:

- preparing grounded context for RAG prompts
- packing evidence for tool-using agents inspired by ReAct-style loops
- red-team analysis of retrieved corpora
- local context audits before moving to a larger retrieval stack
- educational examples for context engineering and safety analysis

The risk scan is especially useful when teams work with heterogeneous corpora assembled from issue trackers, Markdown docs, chat exports, or scraped pages. These sources may contain embedded instructions, secrets, or policy text not meant to flow directly into a model prompt.

7. Strengths and Limitations

The main strengths of Context Forge are:

- small and inspectable implementation
- explicit treatment of budget, diversity, and safety in one method
- no external dependency for the packing stage
- outputs that preserve source identifiers and risk warnings

Its limitations are equally important:

- relevance scoring is lexical rather than embedding-based
- risk scanning is heuristic and pattern-driven
- token estimation is approximate
- no large benchmark evaluation is claimed here

For these reasons, Context Forge should be understood as a practical method layer rather than a replacement for full-scale retrieval infrastructure.

8. Discussion

Context engineering has recently become a catch-all phrase, but the field still lacks many small, inspectable artifacts that connect retrieval choices to actual prompt contents. Much of the literature and tooling conversation emphasizes end-to-end results while leaving the intermediate packing logic under-specified.

Context Forge is useful because it makes that intermediate logic concrete. A reader can inspect how a chunk was formed, why it ranked where it did, whether it was penalized for risk, and how it ended up inside the final prompt block. That visibility makes the method valuable even when teams later replace parts of it with denser retrieval or model-based reranking.

9. Conclusion

Context Forge offers a lightweight method for diversity-aware and risk-aware context packing in retrieval and agent workflows. Its contribution is not scale. It is inspectability. By combining section-aware chunking, transparent ranking, heuristic risk scanning, and budgeted packing, the method gives teams a practical way to reason about the context they hand to models. For local development, evaluation, and safety review, that is a meaningful and reusable contribution.

References

The bundled bibliography provides the current references for context engineering, prompt-injection risk, and agent workflow framing.